CA-1 Project Report

Material Code Harmonisation Across Public Enterprises

Course: CSE 276 – Course Title: to be confirmed

Section: TBC Group: TBC

Student Name	Registration No.	Roll No.
Sindhant Dadwal	12509119	43
Sabuj Mondal	12509118	32

Problem source: Smart India Hackathon (SIH), Problem Statement ID SIH26099

Submitted to: Instructor name: to be confirmed

Date of submission: DD-MM-2026: to be confirmed

Declaration

We declare that this report and the accompanying implementation are our own work, carried out for the CA-1 project of CSE 276. All data, code, papers and websites we used are acknowledged in the References. All results reported here were produced by our own implementation. Where a figure could not be established from the project's own outputs, the report states this explicitly (for example, the evaluation metrics in Chapter 7 are reported as NOT EVALUATED).

Use of AI tools: AI tools were used for project planning, debugging assistance, technical review, annotation assistance and report preparation. In particular, an AI-assisted annotation workflow produced the binary labels and advisory recommendations for the 150 candidate pairs; these are not independent human ground truth and are kept separate from reviewer decisions. We are responsible for every line of code and text submitted.

Name	Registration No.	Signature
Sindhant Dadwal	12509119	
Sabuj Mondal	12509118	

Abstract

Public sector enterprises record the same industrial material under different descriptions and codes, which causes duplicate stock and prevents joint purchasing. This project, inspired by Smart India Hackathon problem statement SIH26099, is an academic precision-oriented prototype for AI-assisted entity resolution of material records. It uses the community-published SIH26099 CPSE Material Codes corpus (21,513 raw descriptions from Oil India, NTPC and IOCL, CC BY 4.0), of which 6,634 form the preprocessed candidate set. The pipeline extracts category-aware engineering attributes, generates candidates with TF-IDF (22,464 pairs from 22,001,661 possible), measures similarity with all-MiniLM-L6-v2 embeddings, detects specification conflicts and clusters records while preventing transitive conflicts. A reviewer web application records human decisions only on an explicit Save & Next. The baseline is exact-string matching, which finds 1,661 pairs, only 15 of them cross-enterprise. The final delivery report records 168 passing software tests, which verify code behaviour, not matching accuracy. Only 20 of 150 candidate pairs carry binary labels, produced by an AI-assisted workflow rather than independent human review, so supervised training stays locked and precision, recall, F1 and cluster purity are NOT EVALUATED. No claim is made that the method outperforms the baseline.

Keywords: entity resolution; material master data; TF-IDF; sentence embeddings; human-in-the-loop review

Table of Contents

Declaration.....2
Abstract.....3
1. Introduction.....5
1.1 Background and Motivation.....5
1.2 Problem Statement.....5
1.3 Source of the Problem.....6
1.4 Who Benefits.....6
2. Objectives.....6
3. Related Work.....7
4. Dataset.....8
4.1 Source and Licence.....8
4.2 Description.....8
4.3 Exploratory Data Analysis.....9
4.4 Preprocessing.....10
5. Methodology.....11
5.1 Overall Approach.....11
5.2 Baseline.....12
5.3 Proposed Method.....12
5.4 Syllabus Mapping.....14
6. Implementation.....15
6.1 Tools and Libraries.....15
6.2 System Architecture.....15
6.3 Demo / User Interface.....16
6.4 How to Run.....17
7. Results and Evaluation.....17
7.1 Evaluation Setup.....17
7.2 Results Against the Baseline.....18
7.3 Error Analysis.....21
7.4 Discussion.....22
8. Limitations and Ethical Considerations.....22
9. Conclusion and Future Work.....23
References.....24
Appendix A: Individual Contribution.....25
Appendix B: Code Repository and Folder Structure.....25
Appendix C: Additional Results.....25

1. Introduction

1.1 Background and Motivation

Large public sector enterprises in India, such as those in the oil, gas and power sectors, buy and store tens of thousands of different engineering items: valves, pumps, cables, bearings, pipes, instruments and consumables. Each enterprise keeps its own material master, a catalogue in which every item has a code and a text description. Because these catalogues were built separately, by different people, over many years, the same physical item can appear under different codes and different words. One enterprise may write “API 6D BALL VALVES”, another “BALL VALVES API 6D”, and a tender may add a quantity, as in “API 6D BALL VALVES QTY 76 NUMBERS”.

This fragmentation has practical costs. Stock of the same item is held in several places without anyone knowing, procurement teams cannot easily pool demand to negotiate better terms, and engineers cannot quickly check whether a spare held by one enterprise could serve another. A common, harmonised code for functionally equivalent materials would remove much of this waste.

Harmonisation is harder than it looks. Two descriptions that read almost the same can refer to technically incompatible items: a valve with a different pressure class, a bolt with a different thread size, or a pump with a different flow rate. Wrongly merging such records is not just a data-quality error; in an industrial plant it could lead to the wrong part being issued. The task therefore needs both flexible text matching, to find records that use different wording, and strict engineering checks, to stop unsafe merges. This is the setting of our project.

1.2 Problem Statement

The problem is listed on the Smart India Hackathon portal as “AI-Driven Standardization and Harmonization of Material Codes Across CPSEs”, proposed by the Ministry of Petroleum and Natural Gas under the Smart Automation category (software) [1]. In summary, the statement asks for a system that recognises when different records describe the same material and proposes one common code, together with reviewer support and an audit trail.

In our own words: given material records from several public enterprises, each with its own code and free-text description, the system must decide which records describe functionally equivalent items. It must do this even when the wording, word order, abbreviations or catalogue formats differ, and it must refuse to merge records whose engineering specifications conflict. For every proposed group of records it must suggest one harmonised identifier and show the evidence for the proposal, so that a human reviewer can approve or reject it.

1.3 Source of the Problem

The problem comes from the Smart India Hackathon (SIH) 2026, problem-statement ID **SIH26099**, proposed by the Ministry of Petroleum and Natural Gas. The official problem statements are published on the SIH portal (https://www.sih.gov.in); the listing we consulted is cited in the References [1].

This project is inspired by SIH26099. It is an academic prototype for a CSE 276 course project and is not a complete solution to the hackathon problem. It has not been deployed by any public sector enterprise, it does not use any enterprise's internal material master, and its output must not be used to authorise real procurement substitutions.

1.4 Who Benefits

If developed further, a system of this kind would help the following groups:

- **Procurement teams** in public enterprises, who could see that apparently different purchase items are the same material and combine demand.
- **Inventory management departments**, who could detect duplicate stock held under different codes.
- **Material master data administrators**, who could clean catalogues faster, because the system proposes merges with evidence instead of requiring a manual search.
- **Engineers**, who can judge technical equivalence more quickly when attribute conflicts such as size or pressure class are highlighted.
- **Procurement analysts**, who need consistent codes for spend analysis across enterprises.
- **The public enterprises themselves**, through lower stock-holding cost and better use of shared purchasing power.

2. Objectives

The three objectives allocated for this project are reproduced below, unchanged.

1. **Build a matching system that groups functionally equivalent material records and proposes one harmonised code with evidence.**

   How it is addressed: the pipeline in Chapter 5 generates candidates with TF-IDF, scores them lexically and semantically, extracts category-aware engineering attributes, detects specification conflicts, makes evidence-based decisions in a precision-first matching engine and groups records with conflict-aware clustering. The final delivery report states that these parts are implemented. *How it will be shown:* by the proposed clusters with their evidence in the reviewer application, and by the reviewer approval rate once independent review is complete.

2. **Produce an entity resolution model with pairwise precision/recall over a labelled sample.**

How it is addressed: a pairwise decision procedure over lexical, semantic and attribute-evidence features (Section 5.3); a Random Forest was considered but is not trained. *How it will be shown:* by pairwise precision and recall on an independently labelled sample. Only 20 confirmed binary labels exist, all from an AI-assisted workflow and below the pilot threshold of 30 balanced binary labels, so this evidence is not yet available (Chapter 7).

3. **Report pairwise precision/recall/F1, cluster purity. Baseline: exact string match. Deliver Reviewer UI showing proposed merges with evidence.**

How it is addressed: an exact-string baseline (Section 5.2), an evaluation plan for precision, recall, F1 and cluster purity (Section 7.1), and the reviewer web application (Section 6.3). *How it will be shown:* a results table comparing the proposed method with the baseline on all four metrics (Table 11 reports the current status as NOT EVALUATED) and a live demonstration of the reviewer application.

3. Related Work

Entity resolution, also called record linkage or entity matching, has a long history. We summarise the approaches that shaped our design and say how our project differs.

Probabilistic record linkage. Fellegi and Sunter formalised linkage as a decision between match, non-match and possible match, based on how well the fields of two records agree [2]. Our three decisions, EQUIVALENT, NOT_EQUIVALENT and NEEDS_REVIEW, follow the same three-way idea. Unlike their method, we do not estimate field-agreement weights from data; our attribute match and conflict indicators are engineered from engineering attributes and used as evidence.

Blocking and filtering. Comparing every record with every other is quadratic, so entity-resolution systems first select candidate pairs cheaply. Papadakis et al. survey blocking and filtering techniques for this purpose [3]. Our TF-IDF candidate generation is a simple similarity-threshold filter of this kind; we do not implement the more advanced schemes in the survey.

TF-IDF lexical similarity. Salton and Buckley's term-weighting work [4] underlies the vector representation we use to find lexically similar descriptions, including with word bigrams so that phrases such as "BALL VALVE" carry weight.

Sentence embeddings. Sentence-BERT produces fixed-size sentence vectors that can be compared by cosine similarity [5], so descriptions with different wording but similar meaning lie close together. We use the compact all-MiniLM-L6-v2 model [6], which is based on the

MiniLM distillation approach [7]. Embeddings capture meaning but not exact numeric specifications, which is why we pair them with attribute checks.

Deep learning for entity matching. Mudgal et al. compare deep-learning designs for entity matching and find them most helpful on textual and dirty data [8]. Such models need many labelled pairs. Our project has no independent labelled set yet, so we deliberately do not train a deep matcher.

Human-in-the-loop matching. The Magellan system treats entity matching as a workflow with sampling, labelling and debugging by a human [9]. We follow the same principle: our reviewer interface lets a person approve or reject each proposed merge, and graph clustering is done with NetworkX [10].

Our work differs from these in being a small academic prototype specific to engineering material descriptions, in treating specification conflicts as a hard stop for automatic merging, and in keeping AI-assisted suggestions separate from human labels.

4. Dataset

4.1 Source and Licence

The project uses the dataset **SIH 26099 — Collected Dataset** (repository name sih26099-cpse-material-codes), published on Hugging Face by the user Prasenjeet25 [11]. The dataset card gives its collection date as 8 September 2026 and its licence as **CC BY 4.0**, which permits reuse with attribution. It is a community-published collection derived from public procurement sources (tender portals of NTPC, IOCL and Oil India), not an official dataset supplied or certified by the enterprises. Some records are tender or procurement-plan descriptions rather than individual material-master entries, and we do not claim that every record has been independently verified.

4.2 Description

The main table of the dataset is a material-description corpus with 21,513 rows, of which 17,400 are distinct description strings [11]. Table 1 summarises its structure and Table 2 gives the number of records per enterprise, both for the raw corpus and for the 6,634 records that form our preprocessed candidate set.

Table 1: Main description corpus of the dataset (from the dataset card)

Property	Value
Raw records	21,513 (17,400 distinct description strings)
Enterprises	Oil India Limited (OIL), NTPC Limited, Indian Oil Corporation Limited (IOCL)
Key columns	corpus_id, organization, source_system, source_section, tender_reference, tender_id, description, description_kind, item_type_hint, quantity, unit, location, product_category, document_url, source_url
Description kinds	e.g. tender_title, historical_tender, procurement_plan_item, doc_spec_string
Labels	None. item_type_hint is a coarse keyword-based type, not a manual annotation
Cross-enterprise duplicates	Near-duplicates across organisations are kept on purpose; only exact duplicates within one organisation and description kind are collapsed
Collection date	8 September 2026

Table 2: Records per enterprise: raw corpus and preprocessed candidate set

Enterprise	Raw records	Processed records	Share of processed
Oil India Limited (OIL)	18,950	5,394	81.3%
NTPC Limited	1,843	732	11.0%
Indian Oil Corporation Limited (IOCL)	720	508	7.7%
Total	**21,513**	**6,634**	**100%**

The raw figures come from the dataset card; the processed figures are the counts recorded by our preprocessing stage. In total 6,634 records (30.8% of the raw corpus) form the processed candidate set and 14,879 records are held back. Held-back records are not deleted: they remain in the raw corpus and can be brought into later experiments. The corpus is strongly imbalanced, with Oil India contributing about 88% of the raw records, which limits how much can be concluded about the smaller enterprises.

4.3 Exploratory Data Analysis

The exploration stage (notebook 01) examines the dataset schema, missing values, enterprise-wise distribution, description lengths and sample descriptions. At the time of writing, the executed outputs of these analyses (length distributions and missing-value counts) are not yet saved in the project repository, so this section reports only the distributional facts that are documented in the dataset card and in our processing counts.

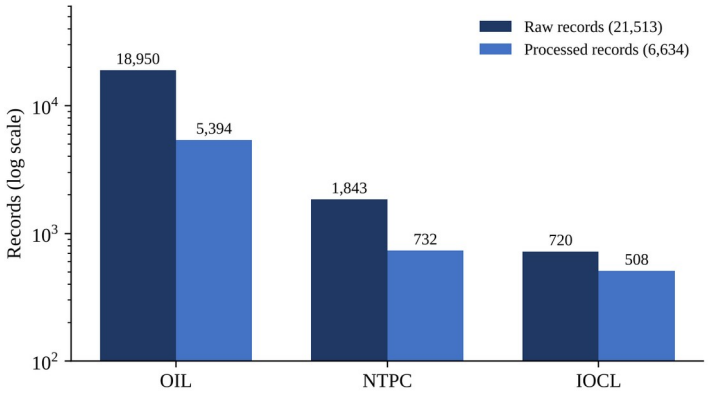


Figure 1: Raw and processed records per enterprise (log scale).

Figure 1 shows that Oil India dominates the raw corpus and that the processing stage keeps a smaller share of its records (28.5%) than of NTPC (39.7%) or IOCL (70.6%). Two data-quality observations from the dataset card matter for the rest of the work: many descriptions are tender titles that may bundle several items, and descriptions can carry quantities (for example “QTY 76 NUMBERS”) that are irrelevant to material identity but defeat exact-string comparison.

4.4 Preprocessing

Preprocessing (notebook 02) turns raw descriptions into a cleaner candidate set. Its steps are text normalisation (casing, whitespace and punctuation), acronym expansion (for example SS becomes STAINLESS STEEL), unit standardisation (for example mm, inch and metre), regular-expression extraction of engineering attributes, record validation, candidate record selection and export of the processed dataset. The notebook’s own test descriptions “Stainless Steel Bolt M10”, “SS Hex Bolt, 10mm” and “Hexagonal Stainless Steel Fastener M10” illustrate the aim: different wordings should normalise toward comparable forms and expose the same attributes.

Table 3: Engineering attributes extracted during preprocessing

Attribute	Meaning and use
Diameter	Nominal size of pipes, valves, fittings or fasteners; a mismatch is a specification conflict
Material grade	Material or alloy grade; a mismatch indicates different material
Pressure class	Rated pressure class of valves, flanges and fittings
Metric thread	Thread designation such as M10 versus M12 for fasteners
Schedule	Pipe wall-thickness schedule
Other specifications	Further engineering specifications where the description contains them

Extracted attributes are supporting evidence, not ground truth. They come from pattern rules, so they can be missing, wrong or ambiguous, and a missing attribute is never treated as a match. We have not yet measured extraction accuracy against a manually checked sample.

Candidate selection and held-back records. Of the 21,513 raw records, 6,634 passed record validation and candidate selection and form the processed set; 14,879 are held back. The criteria that decided which records were held back are applied in the preprocessing code, which is not included in the public repository snapshot, so this report does not list them. **Data split.** No train, validation and test split has been made, because supervised training is intentionally locked until independent human-labelled pairs are available (Section 7.1).

5. Methodology

5.1 Overall Approach

The approach is a staged pipeline from raw records to reviewed harmonised identifiers (Figure 2). Each stage corresponds to one block of the diagram and, in most cases, to one notebook.

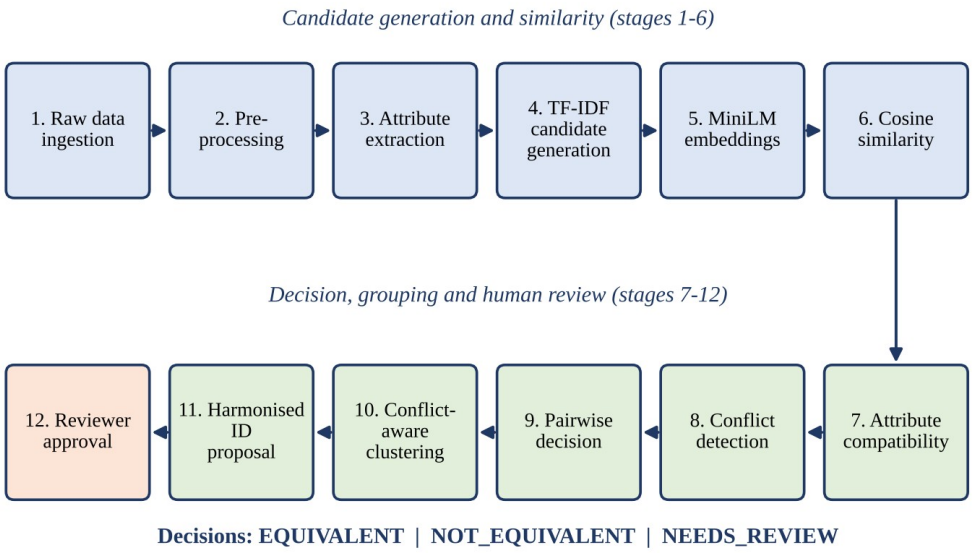


Figure 2: Pipeline from raw data to reviewer approval.

- **Stages 1–3** ingest the corpus, normalise descriptions and extract engineering attributes.
- **Stage 4** uses TF-IDF to select a manageable set of candidate pairs instead of comparing all 22 million possible pairs.
- **Stages 5–6** embed each description with MiniLM and compute cosine similarity for the candidate pairs.
- **Stages 7–9** check attribute compatibility, detect specification conflicts and combine the evidence into one of three decisions.
- **Stages 10–12** group records, propose a harmonised identifier with its evidence and pass the proposal to a human reviewer.

The seven notebooks are seven stages of one pipeline; they are not seven different machine-learning models.

5.2 Baseline

The baseline is **exact-string matching**: two records are declared a match if their description strings are identical. It is simple, fast, needs no training and is the baseline named in the project objectives. It is a fair baseline because it is what a naive deduplication of a catalogue would do, and any more sophisticated method should at least find matches that it misses.

Its weakness is that it cannot see through reordered words, abbreviations or appended quantities. “API 6D BALL VALVES”, “BALL VALVES API 6D” and “API 6D BALL VALVES QTY 76 NUMBERS” may describe the same item, yet no two of these strings are equal. On our processed records the baseline finds 1,661 exact-match pairs (Section 7.2). Notebook 03 is designed to evaluate it on raw and on normalised strings once labelled pairs exist.

5.3 Proposed Method

The proposed method combines lexical candidate generation, semantic embeddings and engineering-attribute evidence. It is built so that no single signal can force a merge.

Lexical candidate generation. Every processed description is converted to a TF-IDF vector [4] using the configuration in Table 4. Pairs with cosine similarity of at least 0.35 become candidates. This reduces the search space from 22,001,661 possible pairs (6,634 × 6,633 / 2) to 22,464 candidates, a reduction of 99.9%. This reduction measures how much work is saved; it says nothing about whether the candidates are correct, and it should not be confused with precision or accuracy.

Semantic similarity. Each description is embedded with all-MiniLM-L6-v2 [6], giving a 384-dimensional vector. For two vectors *u* and *v* the cosine similarity is $\cos(u, v) = (u \cdot v) / (\|u\| \|v\|)$. Descriptions that use different words for similar content, such as reordered phrases, receive high similarity. However, **high semantic similarity does not prove functional equivalence**: notebook 04 notes that purely semantic models cannot capture decisive dimension differences such as M10 versus M12.

Table 4: Configuration of the candidate-generation and embedding stages

Setting	Value
TF-IDF n-gram range	(1, 2)
TF-IDF minimum document frequency	2
TF-IDF sublinear term frequency	True
Candidate similarity threshold	0.35
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Embedding dimensions / layers / max length	384 / 6 / 128 tokens
Embedding matrix shape	(6634, 384), saved as models/candidate_embeddings.npy

Pairwise features. For every candidate pair the evidence is collected as the features in Table 5.

Table 5: Features used to describe a candidate pair

Feature	Purpose
TF-IDF similarity; semantic cosine similarity; similarity difference	Lexical and semantic closeness, and their disagreement
Exact-match indicator	Whether the strings are identical
Attribute conflict indicator; conflict count; confirmation count	How many extracted attributes disagree or agree
Cross-enterprise indicator	Whether the records come from different enterprises
Diameter, material-grade, pressure-class, metric-thread and schedule match	Per-attribute agreement

Decision rules and the classifier. The decisions are EQUIVALENT, NOT_EQUIVALENT and NEEDS_REVIEW (Figure 3). The matching engine (src/matching_engine.py, as reported) is **precision-first**: when evidence is incomplete or ambiguous it returns NEEDS_REVIEW instead of risking a false merge. A confirmed technical conflict prevents an automatic merge; missing attributes are never treated as matching; and identical but generic descriptions do not by themselves establish functional equivalence. A Random Forest classifier [12, 13] was considered as a supervised decision method. **It has not been trained.** The readiness status is NOT_READY_FOR_TRAINING, because the 20 confirmed binary labels are fewer than the pilot threshold of 30 balanced binary labels and come from an AI-assisted workflow, not independent review. Supervised training is therefore deliberately kept locked.

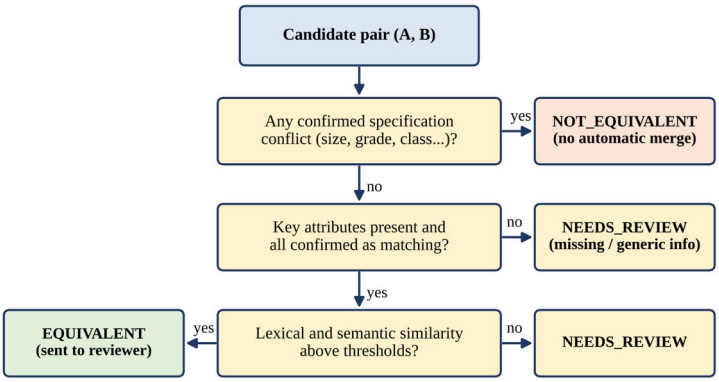


Figure 3: Evidence-based decision logic for a candidate pair.

Technical compatibility by category. What counts as equivalent depends on the kind of material (Table 6). The final delivery report states that category-aware attribute extraction and comparison are implemented in the matching engine. The table lists the attributes that must

agree; the coverage and accuracy of the extraction for each category have not been independently verified, and this report does not claim that every real-world case is handled.

Table 6: Attributes that must agree for functional equivalence, by category

Category	Attributes to compare
Cables	Voltage rating, core count, conductor size, conductor material, insulation, applicable standard
Valves	Valve type, nominal size, pressure class, body material, end connection, applicable standard
Pumps	Pump type, flow rate, head, material, operating service, applicable standard
Bearings	Bearing type, dimensions, series, clearance
Instrumentation	Measurement principle, measurement range, accuracy, application

Conflict-aware clustering and harmonised identifiers. Accepted pairs form a graph whose nodes are records, and connected components (NetworkX [10]) give groups of equivalent records. Plain connected components carry a known risk (Figure 4): if A matches B and B matches C, the component contains A, B and C even if A and C have incompatible specifications. The final delivery report states that src/clustering.py addresses this with **transitive conflict detection**: a group is not accepted if any two of its members conflict, even through an intermediate record. It also reports **ambiguous record isolation**: records whose attributes are too ambiguous to compare reliably are kept apart for review instead of bridging groups. The delivery report records automated tests for transitive conflict prevention. These behaviours are reported, not independently verified here, and passing tests do not mean that every merge on real data is correct. A representative identifier and its evidence are then proposed for each accepted group.

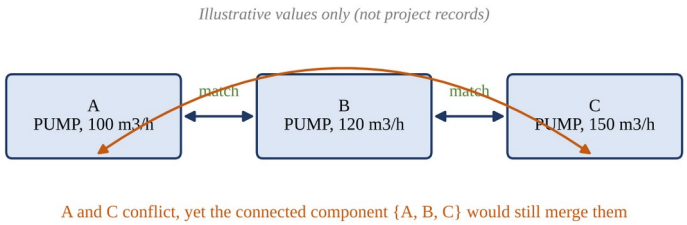


Figure 4: Chain effect in connected-component clustering (illustrative).

5.4 Syllabus Mapping

Table 7 links each technique used in the project to the course unit it comes from.

Table 7: Mapping of project techniques to the course syllabus

Technique used in the project	Course unit	Where used (section / file)
Text normalisation, regular-expression attribute extraction	Unit: to be confirmed	4.4 / notebook 02
TF-IDF vectorisation and cosine similarity	Unit: to be confirmed	5.3 / notebook 03
Exact-string matching baseline	Unit: to be confirmed	5.2 / notebook 03
Sentence embeddings (MiniLM)	Unit: to be confirmed	5.3 / notebook 04
Pairwise classification, Random Forest (not trained)	Unit: to be confirmed	5.3 / notebook 05
Graph clustering (connected components)	Unit: to be confirmed	5.3 / notebook 06
Precision, recall, F1, cluster purity	Unit: to be confirmed	7.1 / notebook 07

6. Implementation

6.1 Tools and Libraries

The project is written in Python and organised as Jupyter notebooks that import helper modules. Table 8 lists the libraries used. The final delivery report records Python 3.13.14 and pytest 8.3.4 on a Windows machine. Versions of the other libraries are to be inserted from the final requirements.txt.

Table 8: Libraries used by the notebooks

Library	Role in the project
pandas, numpy	Data loading, tables and numerical arrays
matplotlib, seaborn	Plots for exploration and evaluation
scikit-learn [13]	TF-IDF vectorisation, classifiers, metrics
sentence-transformers	all-MiniLM-L6-v2 embeddings
networkx [10]	Graph construction and connected components
pytest	Automated tests (version 8.3.4, reported)

6.2 System Architecture

The code is organised as a Python package named src with ten modules: config, data_loader, preprocess, baseline, feature_extraction, model, clustering, evaluation and harmonisation (all imported by the notebooks) and matching_engine (reported in the final delivery report). Precision-first decisions are made in src/matching_engine.py and conflict-aware clustering in src/clustering.py. The combined candidate dataset is data/processed/combined_candidate_pairs_150.csv and the reviewer web application is review_app.py in the project root; notebooks 05, 06 and 07 were updated for this stage (Figure

5). Reviewer decisions are stored separately from AI-assisted suggestions, so that suggestions can never be mistaken for ground truth. The folder tree is in Appendix B.

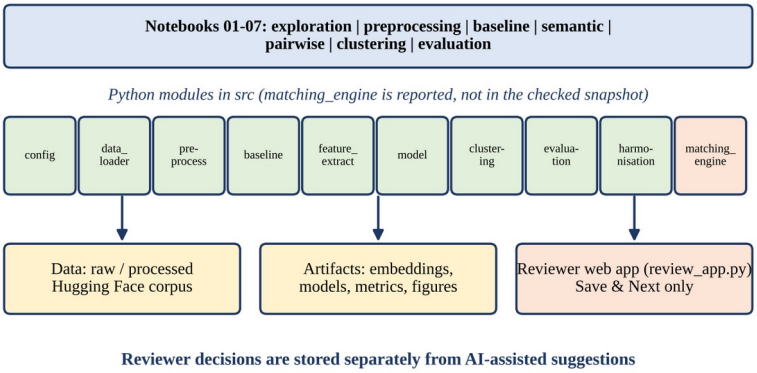


Figure 5: Code organisation and data flow.

6.3 Demo / User Interface

The final delivery report states that the reviewer web application (review_app.py) supports the combined 150-pair dataset and a Batch 1 selection. For each candidate pair it displays the two material descriptions, the enterprise names, the extracted specifications, the source references, the similarity scores and the AI advisory recommendation, together with human review controls for the decision, confidence and notes (Figure 6). Selecting a label neither saves nor moves to the next pair; only the explicit **Save & Next** action persists the review and advances, and entering confidence or notes does not trigger navigation. This prevents accidental labels.

The application code is not part of the public repository snapshot, so these features are reported, not independently verified, and no screenshot is included; Figure 6 is a schematic of the layout, not a screenshot. Real screenshots should be added once the application is run for the demonstration. Human approval reduces incorrect harmonisation because a merge only takes effect after a person has seen the evidence, including any conflicting specifications.

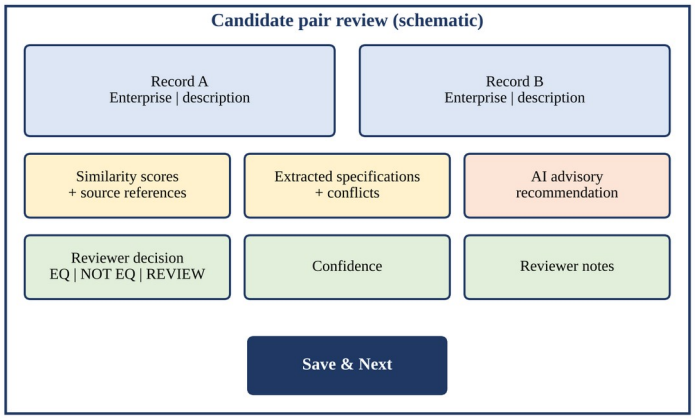


Figure 6: Schematic of the reviewer screen layout (not a screenshot).

6.4 How to Run

The code is at https://github.com/SHIVA07605/CSE-276-PROJECT [14]. To reproduce the work: clone the repository; install the packages listed in requirements.txt; download the dataset from the Hugging Face link in Section 4.1 into the data folder; run notebooks 01 to 07 in order; and start the reviewer application review_app.py as described in the README. Full steps belong in the README of the repository.

Testing. The final delivery report records the automated test result in Table 9. The tests check that the code behaves as designed, including transitive conflict prevention; the number of passing tests is not a measure of matching accuracy. The per-test breakdown has not been checked against saved test output and is therefore not listed.

Table 9: Automated test result reported in the final delivery report

Item	Reported value
Tests passed	168
Tests skipped	1
Tests failed	0
Python / pytest	3.13.14 / 8.3.4
Platform	Windows

7. Results and Evaluation

7.1 Evaluation Setup

The objectives require four metrics: pairwise precision, pairwise recall, pairwise F1-score and cluster purity, each compared against the exact-string baseline. Precision is TP / (TP + FP), recall is TP / (TP + FN) and F1 is their harmonic mean. Cluster purity is $(1/N)\ \Sigma_k \max_j |c_k \cap t_j|$, where c_k are predicted clusters and t_j true entity classes.

These metrics are meaningful only against **independent ground truth**, that is, labels given by human reviewers who were not guided by the model. The project does not yet have enough of them. The combined 150-pair dataset has 20 confirmed binary labels (2 positive, 18 negative), 30 uncertain labels and 100 pending labels, and the confirmed labels come from the existing AI-assisted annotation workflow, not from independent human review. They number fewer than the pilot threshold of 30 balanced binary labels, so the readiness status is NOT_READY_FOR_TRAINING and supervised training is locked. The four metrics are therefore reported as **NOT EVALUATED**. We report no substitute score, and we make no claim that the proposed method outperforms the baseline.

The planned evaluation is: (1) review the 100 pending candidates and obtain further independently verified labels from at least two reviewers, measuring their agreement; (2)

unlock supervised training only when the readiness criteria are met; (3) split pairs by cluster and by material ID, because some materials appear in several pairs (Section 7.2); (4) evaluate the baseline and the proposed method on the same independent test set; (5) compute cluster purity against valid reference clusters; and (6) report the metrics with confidence intervals. Table 10 separates the different kinds of numbers in this report so that they are not confused.

Table 10: Kinds of numbers in this report and their evidential status

Kind	Examples	Status
Candidate-generation statistics	22,464 candidate pairs; 930 cross-enterprise	Measures work reduction, not correctness
Similarity scores	TF-IDF and cosine values	Evidence for a decision, not a decision
AI-assisted labels and suggestions	2 / 18 / 30 / 100 pending labels; 16 / 5 / 79 Batch 2 suggestions	AI-assisted provenance; not independent human ground truth
Software test results	168 passed, 1 skipped, 0 failed	Reported; verifies code behaviour, not matching accuracy
Model predictions	Random Forest output	None: classifier not trained
Independent ground-truth evaluation	Precision, recall, F1, purity	NOT EVALUATED

7.2 Results Against the Baseline

Table 11 is the results table required by the objectives. Because no independent ground truth exists, every metric is NOT EVALUATED, and no value is shown in bold as best.

Table 11: Results against independent ground truth (not yet evaluated)

Model	Precision	Recall	F1	Cluster purity
Baseline: exact-string match	NOT EVALUATED	NOT EVALUATED	NOT EVALUATED	NOT EVALUATED
Proposed: TF-IDF + MiniLM + attribute evidence	NOT EVALUATED	NOT EVALUATED	NOT EVALUATED	NOT EVALUATED

What can be reported are the actual counts produced by the baseline and by candidate generation, shown separately. Table 12 gives the exact-match statistics on the 6,634 processed records.

Table 12: Exact-string baseline statistics on the processed records

Statistic	Value
Processed records	6,634
Repeated descriptions	1,177
Exact-match pairs	1,661
Intra-enterprise exact-match pairs	1,646 (99.1%)
Cross-enterprise exact-match pairs	15 (0.9%)

Almost all exact matches (1,646 of 1,661) are repeats within one enterprise. Only 15 connect different enterprises, so exact matching is of little use for the cross-enterprise harmonisation that the problem requires. Table 13 gives the candidate-generation statistics.

Table 13: TF-IDF candidate-generation statistics (threshold 0.35)

Statistic	Value
Possible pairs among 6,634 records	22,001,661
Candidate pairs	22,464 (0.10% of possible pairs)
Intra-enterprise candidate pairs	21,534 (95.9%)
Cross-enterprise candidate pairs	930 (4.1%)
Pairs with an attribute conflict	1,540 (6.9%)
Pairs with no conflict or missing attributes	20,924 (93.1%)

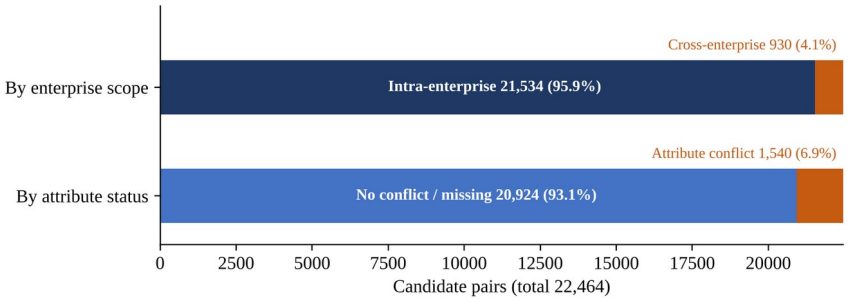


Figure 7: Composition of the 22,464 candidate pairs.

These statistics describe how the search space was narrowed; they are not accuracy figures. The candidate pairs selected for annotation are held in the combined dataset data/processed/combined_candidate_pairs_150.csv, which contains 50 Batch 1 pairs and 100 Batch 2 pairs. Its current label status is in Table 14.

Table 14: Label status of the combined 150-pair candidate dataset

Status	Pairs	Provenance
Positive (confirmed binary label)	2	AI-assisted workflow
Negative (confirmed binary label)	18	AI-assisted workflow
Uncertain	30	AI-assisted workflow
Pending	100	Awaiting review
Total	**150**	

The 20 confirmed binary labels (2 positive and 18 negative) come from the existing AI-assisted annotation workflow. They are **not independent human ground truth** and must not be used as a test set. They are fewer than the pilot threshold of 30 balanced binary labels. The earlier Batch 1 suggestion counts (2 positive, 17 negative, 31 uncertain, Table 15) and the latest combined counts differ by one pair; we report both as given and do not reconcile them without the label file.

Batch overlap. Batch 2 contains 100 unique candidate pairs, and there are zero duplicate unordered pairs between Batch 1 and Batch 2. This is pair-level uniqueness. It is not the same as endpoint-level disjointness: 11 material IDs from Batch 1 reappear in Batch 2 across 38 pairs, so the two batches share some records. Duplicated pairs would inflate a test set directly; shared endpoints can still leak information between training and test pairs, which is why the evaluation plan splits by material ID as well as by cluster. Batch 2 was designed to widen coverage across industrial categories, using IOCL–OIL (73), NTPC–OIL (15) and IOCL–NTPC (12) pairs (Appendix C). Table 15 gives the earlier advisory suggestions per batch.

Table 15: AI-assisted advisory suggestions per batch, from the earlier status report (not human labels)

Suggestion	Batch 1	Batch 2	Total
Tentative positive	2	16	18
Tentative negative	17	5	22
Uncertain	31	79	110
Total pairs	**50**	**100**	**150**

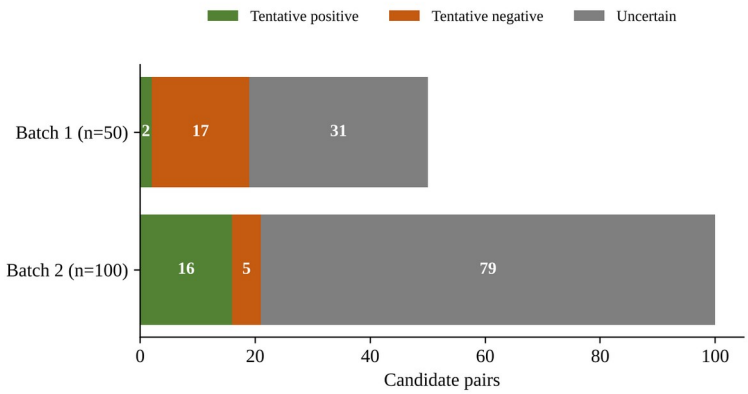


Figure 8: AI-assisted advisory suggestions per batch.

The human review fields for the 100 pending pairs remain empty until reviewers have examined them.

7.3 Error Analysis

Without ground truth we cannot count false positives and false negatives, so the analysis is qualitative and based on the statistics above.

- **Baseline misses.** Reordered or quantity-suffixed descriptions such as the three API 6D ball-valve variants in Section 5.2 are not equal as strings, so the baseline cannot link them. The very low cross-enterprise yield (15 pairs) is consistent with enterprises using different phrasing.
- **Risk of false merges.** Of the 22,464 candidates, 1,540 (6.9%) carry an attribute conflict even though their text is similar enough to pass the 0.35 threshold. These are exactly the pairs a text-only method would be tempted to merge, and they show why conflict detection is a hard stop.
- **Semantic similarity is not equivalence.** Descriptions that differ only in a decisive number, such as a bolt of thread M10 versus M12, can be close in embedding space.
- **Ambiguous evidence.** In the combined dataset 30 of the 50 labelled pairs are uncertain, and in the earlier advisory suggestions 110 of 150 (73%) were uncertain (62% in Batch 1, 79% in Batch 2). Many descriptions simply do not contain enough information to decide, which is why NEEDS_REVIEW is a distinct decision and why the engine is precision-first.
- **Extraction limits.** Pattern-based attribute extraction can miss attributes written in unusual formats or misread numbers. Its accuracy has not yet been measured.
- **Chain merges.** Plain connected components can group records that conflict with each other through an intermediate record (Figure 4). The delivery report states that conflict-aware clustering prevents this and that tests cover it; its effect on real candidate groups has not been measured here.

7.4 Discussion

The project is best described as an implemented, precision-oriented matching prototype with an operational reviewer workflow, whose supervised evaluation is still pending. **Objective 1** (group equivalent records and propose a harmonised code with evidence) is addressed by the reported matching engine, conflict-aware clustering and reviewer application, but the quality of the proposed groups has not been judged against independent labels. **Objective 2** (pairwise precision and recall on a labelled sample) is not yet met, because the independent labelled sample does not exist. **Objective 3** (baseline comparison on precision, recall, F1 and cluster purity, plus a reviewer interface) is partly met: baseline statistics are reported and the interface is reported as implemented, but the metrics are NOT EVALUATED.

What worked: the TF-IDF filter reduces the pair space by 99.9%, the conflict flag identifies a visible share of risky pairs, and the reported test suite (168 passed) gives confidence that the code behaves as designed. What did not: exact matching yields few cross-enterprise pairs, and the large share of uncertain or pending pairs shows that the bottleneck is independent labelled data, not the algorithm. We do not claim that the industrial matching problem is solved or that every merge is correct.

8. Limitations and Ethical Considerations

- **Data quality.** The dataset is community-published and derived from public tenders, not certified by the enterprises. Descriptions may be tender titles rather than material-master entries, and not every record has been verified.
- **Incomplete attributes.** Many descriptions lack the attributes needed to decide equivalence; extracted attributes are evidence, not ground truth.
- **False positives and industrial safety.** A wrong merge could lead to an incompatible part being issued. For this reason a confirmed conflict blocks automatic merging and every proposal needs human approval. The prototype must not be used to authorise real procurement substitutions.
- **Evaluation limits.** No independent human labels exist, so precision, recall, F1 and cluster purity are NOT EVALUATED. The 20 confirmed binary labels come from an AI-assisted workflow and are below the pilot threshold; they must not be used as test labels. Passing software tests do not measure matching accuracy.
- **Enterprise imbalance.** Oil India provides about 88% of raw records and 81% of processed records, so results may not generalise to the other enterprises.
- **Clustering.** Conflict-aware clustering and transitive conflict prevention are reported as implemented and tested, but their behaviour on real candidate groups has not been independently verified.

- **Batch overlap.** The two annotation batches have no duplicate pairs, but 11 material IDs recur across 38 pairs, so evaluation splits must account for shared endpoints.
- **Reported, not verified.** The matching engine, clustering code, reviewer application and test results are reported in the final delivery report; they are not in the public repository snapshot checked for this report.
- **Privacy and misuse.** The data come from public procurement sources and describe materials, not individuals. Tender metadata may still identify suppliers, which should be considered before any publication of outputs.
- **Human approval.** Reviewer decisions, confidence and notes should be stored with an audit trail, and reviewers should be able to override the system.

9. Conclusion and Future Work

We built an implemented, precision-oriented prototype for AI-assisted entity resolution of material records, inspired by SIH26099, with an operational reviewer workflow. On a processed set of 6,634 records from a 21,513-record public corpus, the exact-string baseline finds 1,661 pairs, only 15 of them across enterprises, while TF-IDF candidate generation narrows 22,001,661 possible pairs to 22,464 and flags 1,540 with attribute conflicts. The reported matching engine combines category-aware attributes, conflict detection and three-way decisions, and conflict-aware clustering prevents transitive conflicts; 168 software tests are reported as passing. Supervised model evaluation remains pending: only 20 confirmed binary labels exist, they are AI-assisted, and the readiness status is NOT_READY_FOR_TRAINING. The proposed method is therefore not scored and is not claimed to outperform the baseline.

Next steps:

- Review the 100 pending candidate pairs and obtain sufficient independently verified labels, with two reviewers per pair, and measure agreement.
- Unlock supervised training only when the readiness criteria are satisfied (at least 30 balanced binary labels), then consider training the Random Forest.
- Evaluate precision, recall and F1 for the baseline and the proposed method on an appropriate independent test set, split by cluster and by material ID, and evaluate cluster purity using valid reference clusters.
- Measure the accuracy of attribute extraction on a manually checked sample.
- Test the conflict-aware clustering on real candidate groups and report how often it blocks or splits merges.
- Extend and validate the category-specific rules for cables, valves, pumps, bearings and instrumentation.

- Record screenshots of the reviewer application and add an audit trail of reviewer decisions.
- Complete the advanced extension named in the project allocation, if it was not completed.

References

[1] Ministry of Petroleum and Natural Gas, "SIH26099: AI-Driven Standardization and Harmonization of Material Codes Across CPSEs," Smart India Hackathon 2026 problem-statement listing. [Online]. Available: https://www.sihbuddy.in/ps/SIH26099

[2] I. P. Fellegi and A. B. Sunter, "A theory for record linkage," J. Amer. Statist. Assoc., vol. 64, no. 328, pp. 1183–1210, 1969.

[3] G. Papadakis, D. Skoutas, E. Thanos, and T. Palpanas, "Blocking and filtering techniques for entity resolution: A survey," ACM Comput. Surv., vol. 53, no. 2, art. 31, 2020, doi: 10.1145/3377455.

[4] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Inf. Process. Manage., vol. 24, no. 5, pp. 513–523, 1988.

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, Hong Kong, China, 2019, pp. 3982–3992.

[6] Sentence-Transformers, "all-MiniLM-L6-v2," Hugging Face model card. [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[7] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 5776–5788.

[8] S. Mudgal et al., "Deep learning for entity matching: A design space exploration," in Proc. ACM SIGMOD Int. Conf. Manage. Data, Houston, TX, USA, 2018, pp. 19–34.

[9] P. Konda et al., "Magellan: Toward building entity matching management systems," Proc. VLDB Endow., vol. 9, no. 12, pp. 1197–1208, 2016.

[10] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in Proc. 7th Python in Science Conf. (SciPy), Pasadena, CA, USA, 2008, pp. 11–15.

[11] Prasenjeet25, "SIH 26099 — Collected Dataset (sih26099-cpse-material-codes)," Hugging Face Datasets, collected 8 Sep. 2026, licence CC BY 4.0. [Online]. Available: https://huggingface.co/datasets/Prasenjeet25/sih26099-cpse-material-codes

[12] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[13] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[14] S. Dadwal and S. Mondal, "CSE-276-PROJECT," GitHub repository, commit 1f3c050, 30 Oct. 2026. [Online]. Available: https://github.com/SHIVA07605/CSE-276-PROJECT

Appendix A: Individual Contribution

The division of work between the two members must be recorded here from the project records. It has not been established from the repository history, so the entries below are left for confirmation by the members rather than estimated.

Member	Main contributions (code, experiments, report sections, slides)	Share (%)
Sindhant Dadwal (12509119)	To be confirmed by the member	TBC
Sabuj Mondal (12509118)	To be confirmed by the member	TBC

Appendix B: Code Repository and Folder Structure

GitHub repository: https://github.com/SHIVA07605/CSE-276-PROJECT [14]. The tree below was checked against the repository at commit 1f3c050 (30 October 2026).

```
CSE-276-PROJECT/
|-- README.md
`-- notebooks/
    |-- 01_data_exploration.ipynb
    |-- 02_data_preprocessing.ipynb
    |-- 03_baseline_matching.ipynb
    |-- 04_semantic_matching.ipynb
    |-- 05_pairwise_classification.ipynb
    |-- 06_clustering.ipynb
    `-- 07_model_evaluation.ipynb
```

The checked snapshot does not yet contain the following paths, which the final delivery report lists: src/matching_engine.py, src/clustering.py, review_app.py, data/processed/combined_candidate_pairs_150.csv and the updated notebooks 05, 06 and 07, together with the automated tests. The guidelines additionally require requirements.txt, app/, report/ and presentation/. They are to be pushed and this tree updated before final submission.

Appendix C: Additional Results

Table 16 lists the enterprise pairs in Batch 2 and Figure 9 its industrial categories, both counted from the batch description (data/processed/candidate_pairs_batch2_expansion_100.csv). The figures are the team’s recorded counts.

Table 16: Enterprise pairs in the Batch 2 expansion (100 pairs)

Enterprise pair	Candidate pairs
IOCL – OIL	73
NTPC – OIL	15
IOCL – NTPC	12
Total	**100**

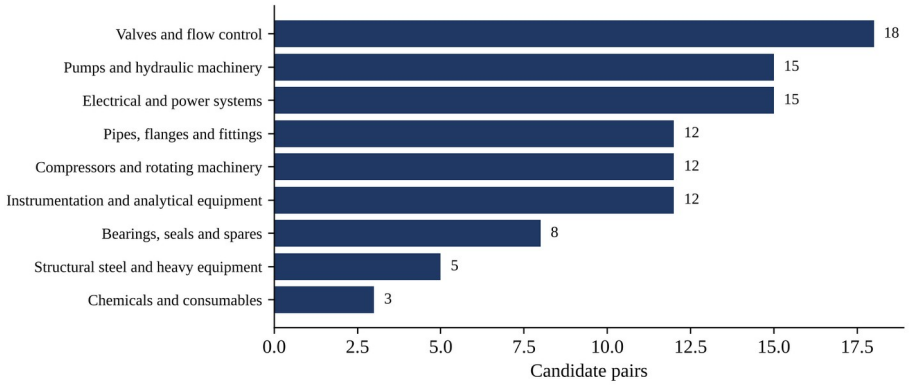


Figure 9: Industrial categories covered by the Batch 2 expansion.